

- 4. Action Plans & Technical Architecture
 - Individual Action Plans
 - 4.1 Project Management & Backend Design Plan (Ander)
 - 4.2 Backend Implementation Plan (Oihan & Julen)
 - 4.3 Design & UI Planning (Liz & Mius)
 - 4.4 Frontend Interactivity Plan (Julen, Oihan, Liz, Mius)
 - Technical Overview (Web-App As Designed)
 - Content for Presentation Slides (Task 4)

4. Action Plans & Technical Architecture

Individual Action Plans

Although significant portions of the project (such as the database of nearly 600 games and the basic PHP/MySQL connection) are already established, each role has a structured plan to ensure the final application is robust and fully functional.

4.1 Project Management & Backend Design Plan (Ander)

- **Objective:** Maintain a clear roadmap and ensure a solid database foundation.
- **Steps:**
 1. Define sprint goals and assign Trello/Jira tasks.
 2. Finalize the `videoGames`, `users`, and `sessions` table schemas.
 3. Lead code review sessions to ensure the frontend variables correctly map to the backend PHP scripts.
 4. Write the final integration test plan.

4.2 Backend Implementation Plan (Oihan & Julen)

- **Objective:** Develop the logic that powers the "AI Recommender".
- **Steps:**

1. Create the REST API endpoint (`recommend.php`) that receives the user's quiz payload via POST.
2. Write the **Hardware Filtering** algorithm to eliminate incompatible games.
3. Write the **Preference Scoring** algorithm (assigning +2 for Time match, +3 for Vibe match).
4. Sort the processed games and return a JSON array containing the Top 10 Results.

4.3 Design & UI Planning (Liz & Mius)

- **Objective:** Deliver a premium, gamer-oriented visual experience.
- **Steps:**
 1. Establish a design system (Color Palette: Slate 900 base, Violet/Cyan accents).
 2. Create high-fidelity wireframes based on Task 2 descriptions.
 3. Build reusable Tailwind CSS components for the Dashboard, Quiz Cards, and Game detail Modals.
 4. Implement CSS transition animations (fade, slide up) to mimic a native app feel.

4.4 Frontend Interactivity Plan (Julen, Oihan, Liz, Mius)

- **Objective:** Stitch the design and backend together into a seamless flow.
- **Steps:**
 1. Initialize Vanilla JS state management to track the quiz progress linearly.
 2. Handle form submission asynchronously with JavaScript `fetch()`.
 3. Simulate AI processing by rendering a loading animation for 3–5 seconds before resolving the fetch promise.
 4. Dynamically append the Top 10 game cards (Image, Score, Description) into the Document Object Model (DOM).

Technical Overview (Web-App As Designed)

- **Core Stack:** HTML5, Tailwind CSS, Vanilla JS for the front-end. PHP and MySQL for the back-end.
- **Database Architecture:**
 - `video_games_quiz_db`: Contains the `videoGames` table populated with nearly 600 curated games.
 - Linked tables include `genres` and `platforms` for relational integrity.
- **Recommendation Logic Flow:**
 1. **Input:** User selects Nintendo Switch, 1h Time, Relaxing Mood.
 2. **Filter:** Query removes PC exclusives or PS5 exclusives.
 3. **Score:** Games matching "1h Time" and "Relaxing" gain points.
 4. **Output:** The system sorts by score and returns the top 10.

Content for Presentation Slides (Task 4)

- **Slide 4.1: Technical Stack & Database:** Explain what the app is built on, highlighting the 600-game MySQL database.
- **Slide 4.2: The Recommendation Algorithm:** A visual flowchart of how the logic works (Filter -> Score -> Sort).
- **Slide 4.3: Design Plan:** Focus on the steps Liz and Mius take to ensure a modern, accessible interface.
- **Slide 4.4: Development Plan:** Summarize how Ander, Oihan, and Julen build the backend endpoints and connect the data.